



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

CAPÍTULO 5: FUNCIONES, REFERENCIAS Y PUNTEROS



César Villacís, Margarita Zambrano, Sofía Bayona, Luis Pastor, David Alvarado, Luis García, Kevin Topón

Universidad de las Fuerzas Armadas – ESPE
Quito, Ecuador

e-mail: {cjvillacis, mezambrano}@espe.edu.ec

Contenido

- **Introducción**
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Introducción

- En este capítulo se aprenderá a diseñar funciones con algoritmos y a utilizar la computadora como una herramienta para resolver problemas creando programas con funciones, referencias y punteros en el lenguaje C/C++. Para la solución de problemas de programación se aplican aspectos de Ingeniería de Software y el método científico que permiten identificar a la programación con un enfoque sistemático.
- En un programa estructurado el flujo lógico se puede optimizar con el uso de funciones, referencias y punteros que es parte de la programación modular. El estudio de las funciones se realiza en base al paso de parámetros por valor, por referencia y por dirección. La aplicación de funciones para crear programas se lo hace con un lenguaje estructurado y modular como es el Lenguaje C/C++.



Figura 1. La importancia de enseñar Lenguaje C y C++.
Fuente: <https://www.iberestudios.com/noticias/por-que-las-universidades-siguen-ensenando-c-y-c/>

Contenido

- Introducción
- **Objetivos**
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Objetivos

- Revisar los principales conceptos en torno a funciones con algoritmos y con estructuras de control.
- Entender cómo trabajan las funciones, las referencias, los punteros y cómo se aplican con el lenguaje C/C++ en la construcción de programas.
- Aprender a manejar el paso de parámetros por valor, por referencia y por dirección con algoritmos en Pseudocódigo y en Lenguaje C/C++.
- Diseñar y crear programas en lenguaje C/C++ utilizando funciones, referencias y punteros. ▪ Resolver casos de estudio con funciones, referencias y punteros aplicados al área matemática.

Contenido

- Introducción
- Objetivos
- **Función**
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Función

- Una función es un mini programa dentro de un programa. Las funciones contienen varias sentencias bajo un solo nombre, que un programa puede utilizar una o más veces para ejecutar dichas sentencias (Granizo, E., 2016). Una función es una unidad de código diseñada para ejecutar una cierta tarea. Una función típica recibe algo de información y retorna algo de información. El concepto de una función es algo similar a una función matemática. Considere el caso de la función $\text{sen}(x)$, la cual toma un parámetro ' x ' y evalúa o retorna algún valor, esto es el seno de ' x '. La utilidad de las funciones puede ser fácilmente demostrada si consideramos un programa sin el uso de ellas, como el siguiente:

Figura 1.1. Ejemplo de un Programa en C/C++.

Contenido

- Introducción
- Objetivos
- Función
- **Problema 5.1**
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Problema 5.1

- Escribir un programa sin utilizar funciones que calcule el área de un rectángulo para los siguientes valores de ancho y largo: a) ancho: 7; largo: 2; b) ancho: 9; largo: 5; c) ancho: 8; largo: 4; ingresados desde el teclado en forma seguida, es decir, uno a continuación de otro.

```
// Librerías.
#include <iostream>
#include <cstdlib>

using namespace std;

// Función principal.
int main()
{
    // Declaración de variables.
    float ancho; // Entrada: ancho del rectángulo.
    float largo; // Entrada: largo del rectángulo.
    float area; // Salida: área del rectángulo.

    // Imprimir un mensaje de información.
    cout << "Área de un rectángulo." << endl;
    // Leer el valor del ancho del rectángulo.
    cout << "Ingrese el valor del ancho: "; cin >> ancho;
    // Leer el valor del largo del rectángulo.
    cout << "Ingrese el valor del largo: "; cin >> largo;

    // Calcular el valor del área del rectángulo.
    area = ancho * largo;
    // Imprimir el valor del área del rectángulo.
    cout << "El área es: " << area << endl;

    // Realizar otros cálculos.
    cout << endl << "Realizar otros cálculos..." << endl << endl;

    // Imprimir un mensaje de información.
    cout << "Área de un rectángulo." << endl;
    // Leer el valor del ancho del rectángulo.
    cout << "Ingrese el valor del ancho: "; cin >> ancho;
    // Leer el valor del largo del rectángulo.
    cout << "Ingrese el valor del largo: "; cin >> largo;

    // Calcular el valor del área del rectángulo.
    area = ancho * largo;
```

Problema 5.1

- Escribir un programa sin utilizar funciones que calcule el área de un rectángulo para los siguientes valores de ancho y largo: a) ancho: 7; largo: 2; b) ancho: 9; largo: 5; c) ancho: 8; largo: 4; ingresados desde el teclado en forma seguida, es decir, uno a continuación de otro.

```
// Imprimir el valor del area del rectangulo.
cout << "El área es: " << area << endl;

// Imprimir un salto de línea (INTRO).
cout << endl;

// Imprimir un mensaje de información.
cout << "Área de un rectángulo." << endl;
// Leer el valor del ancho del rectángulo.
cout << "Ingrese el valor del ancho: "; cin >> ancho;
// Leer el valor del largo del rectángulo.
cout << "Ingrese el valor del largo: "; cin >> largo;

// Calcular el valor del área del rectángulo.
area = ancho * largo;
// Imprimir el valor del área del rectángulo.
cout << "El área es: " << area << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}
```

Programa 5.1. Código del Programa. Fuente: Zambrano M., et al., 2025.

Problema 5.1

- Escribir un programa sin utilizar funciones que calcule el área de un rectángulo para los siguientes valores de ancho y largo: a) ancho: 7; largo: 2; b) ancho: 9; largo: 5; c) ancho: 8; largo: 4; ingresados desde el teclado en forma seguida, es decir, uno a continuación de otro.

```
Area de un rectángulo.  
Ingrese el valor del ancho: 7  
Ingrese el valor del largo: 2  
El Área es: 14  
  
Realizar otros cálculos...  
  
Area de un rectángulo.  
Ingrese el valor del ancho: 9  
Ingrese el valor del largo: 5  
El Área es: 45  
  
Area de un rectángulo.  
Ingrese el valor del ancho: 8  
Ingrese el valor del largo: 4  
El Área es: 32  
Press any key to continue . . .
```

Salida 5.1. Salida del Programa. Fuente: Zambrano M., et al., 2025.

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- **Estructura de una Función**
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Estructura de una Función

De acuerdo con (Joyanes Aguilar, L., Zahonero Martínez, I., 2005), para definir o crear una función en C/C++ se deben considerar los siguientes puntos:

A) Tipo de retorno: Es el tipo de valor devuelto por la función (el valor que la función evalúa) o la palabra reservada “void” si la función no devuelve ningún valor.

B) Nombre: Es el identificador al cual hace referencia la función.

C) Lista de Parámetros: Son los valores que se toman como entrada. Esta lista de declaraciones de los parámetros de la función van separadas por comas.

D) Cuerpo: Es el código que se ejecuta cuando la función es invocada y puede consistir de variables locales, sentencias ejecutables y la palabra reservada “return” acompañada por una expresión lógica, relacional o matemática, siempre y cuando tenga un tipo de retorno la función

Estructura de una Función



Figura 5.1 Estructura de una función; a) Función Potencia(); b) Función FarenheitACelcius(). Fuente: Zambrano M., et al., 2025.

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- **Implementación de una Función**
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Implementación de una Función

De acuerdo con (Granizo, E., 2016), la siguiente sintaxis despliega la forma de la definición o implementación de una función:

Sintaxis:

```
Tipo_de_retorno Nombre_Función(Lista_de_Parámetros)
{
    // Cuerpo de la función
    // Variables locales
    // Sentencias ejecutables
    // return(expresión)
}
```

Implementación de una Función

La Figura 5.2 muestra un ejemplo de la sintaxis de cómo trabaja una función, cuando se la ha definido:

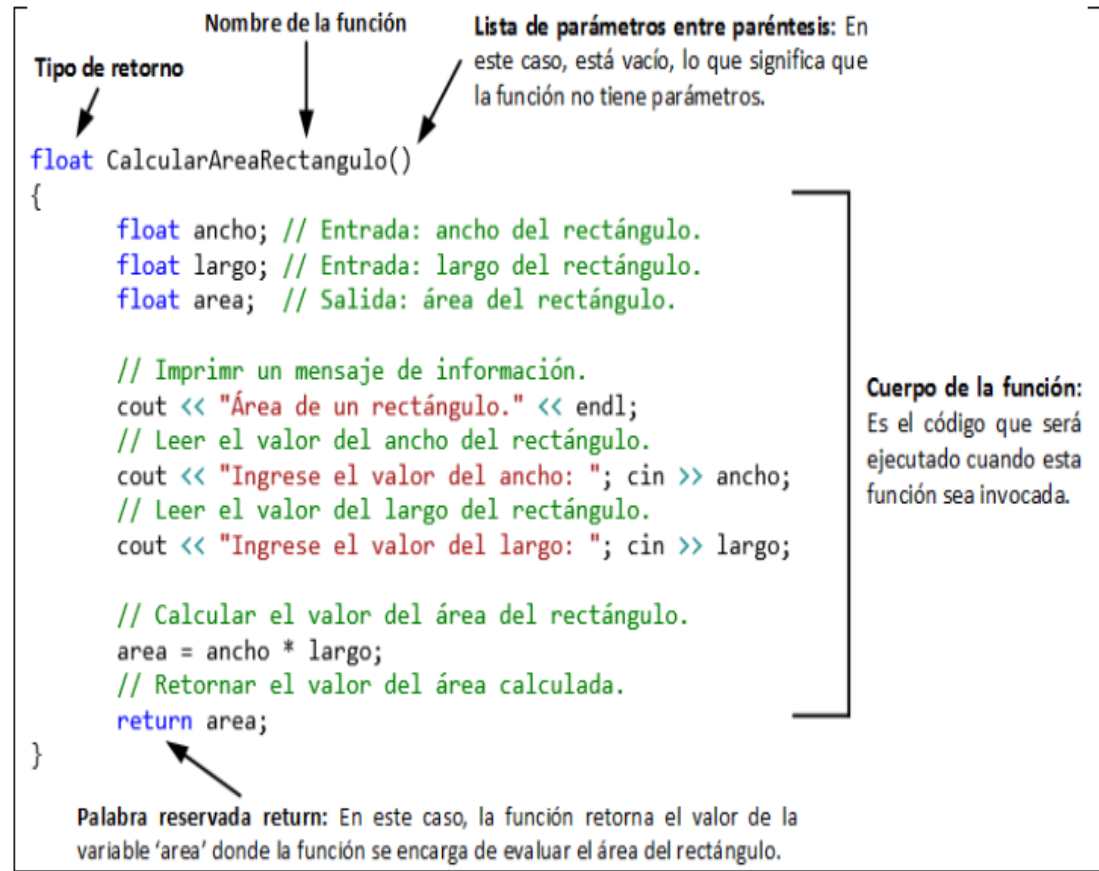


Figura 5.1 Definición de una función Fuente: Zambrano M., et al., 2025.

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- **Problema 5.2**
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Problema 5.2

Escribir un programa utilizando la definición de una función, de tal manera que se calcule el área de un rectángulo para los siguientes valores de ancho y largo: a) ancho: 7; largo: 2; b) ancho: 9; largo: 5; c) ancho: 8; largo: 4; ingresados desde el teclado en forma seguida, es decir, uno a continuación de otro.

```
// Librerías.
#include <iostream>
#include <cstdlib>

using namespace std;

// Esta función lee el valor del ancho y del largo del rectángulo,
// calcula el valor del área del rectángulo y retorna el valor del
// cálculo del área del rectángulo.
float CalcularAreaRectangulo()
{
    float ancho; // Entrada: ancho del rectángulo.
    float largo; // Entrada: largo del rectángulo.
    float area; // Salida: área del rectángulo.

    // Imprimir un mensaje de información.
    cout << "Área de un rectángulo." << endl;
    // Leer el valor del ancho del rectángulo.
    cout << "Ingrese el valor del ancho: "; cin >> ancho;
    // Leer el valor del largo del rectángulo.
    cout << "Ingrese el valor del largo: "; cin >> largo;

    // Calcular el valor del área del rectángulo.
    area = ancho * largo;
    // Retornar el valor del área calculada.
    return area;
}
```

Problema 5.2

Escribir un programa utilizando la definición de una función, de tal manera que se calcule el área de un rectángulo para los siguientes valores de ancho y largo: a) ancho: 7; largo: 2; b) ancho: 9; largo: 5; c) ancho: 8; largo: 4; ingresados desde el teclado en forma seguida, es decir, uno a continuación de otro.

```
// Función principal.
int main()
{
    // Llamada a la función CalcularAreaRectangulo().
    cout << "El área es : " << CalcularAreaRectangulo() << endl;
    // Imprimir un salto de línea (INTRO).
    cout << endl;
    // Realizar otros cálculos.
    cout << "Realizar otros calculos..." << endl << endl;
    // Llamada a la función CalcularAreaRectangulo().
    cout << "El área es : " << CalcularAreaRectangulo() << endl;
    // Imprimir un salto de línea (INTRO).
    cout << endl;
    // Llamada a la función CalcularAreaRectangulo().
    cout << "El área es : " << CalcularAreaRectangulo() << endl;
    // Incorporar una pausa en el programa.
    system("pause");
    return 0;
}
```

Problema 5.2

Escribir un programa utilizando la definición de una función, de tal manera que se calcule el área de un rectángulo para los siguientes valores de ancho y largo: a) ancho: 7; largo: 2; b) ancho: 9; largo: 5; c) ancho: 8; largo: 4; ingresados desde el teclado en forma seguida, es decir, uno a continuación de otro.

```
// Función principal.  
int main()  
{  
    // Llamada a la función CalcularAreaRectangulo().  
    cout << "El área es : " << CalcularAreaRectangulo() << endl;  
    // Imprimir un salto de línea (INTRO).  
    cout << endl;  
    // Realizar otros cálculos.  
    cout << "Realizar otros calculos..." << endl << endl;  
    // Llamada a la función CalcularAreaRectangulo().  
    cout << "El área es : " << CalcularAreaRectangulo() << endl;  
    // Imprimir un salto de línea (INTRO).  
    cout << endl;  
    // Llamada a la función CalcularAreaRectangulo().  
    cout << "El área es : " << CalcularAreaRectangulo() << endl;  
    // Incorporar una pausa en el programa.  
    system("pause");  
    return 0;  
}
```

Programa 5.2. Código del Programa. Fuente: Zambrano M., et al., 2025.

Problema 5.2

Escribir un programa utilizando la definición de una función, de tal manera que se calcule el área de un rectángulo para los siguientes valores de ancho y largo: a) ancho: 7; largo: 2; b) ancho: 9; largo: 5; c) ancho: 8; largo: 4; ingresados desde el teclado en forma seguida, es decir, uno a continuación de otro.

```
Área de un rectángulo.  
Ingrese el valor del ancho: 7  
Ingrese el valor del largo: 2  
El Área es : 14  
  
Realizar otros calculos...  
  
Área de un rectángulo.  
Ingrese el valor del ancho: 9  
Ingrese el valor del largo: 5  
El Área es : 45  
  
Área de un rectángulo.  
Ingrese el valor del ancho: 8  
Ingrese el valor del largo: 4  
El Área es : 32  
Press any key to continue . . .
```

Salida 5.2. Salida del Programa. Fuente: Zambrano M., et al., 2025.

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- **Problemas que Resuelven las Funciones**
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Problemas que Resuelven las Funciones

De acuerdo con (Deitel, H., Deitel, P., 2003), los problemas más importantes que resuelven las funciones son:

- A) Duplicación del código: Al utilizar funciones ya no es necesario volver a escribir tanto código de nuevo, como en el ejercicio 5.2 a diferencia del ejercicio 5.1 que necesita mucho más código.
- B) Reutilización del código: Las funciones permiten utilizar una y otra vez y las veces que se requiera una función solo invocándola o llamándola desde cualquier parte de un programa.
- C) Mantenición del código: El uso de funciones permite dar mantenimiento y actualizar el código de un programa más fácilmente.

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- **Uso de Funciones**
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Uso de Funciones

De acuerdo con (Luna, F., 2004), para utilizar adecuadamente las funciones dentro de los programas se deben considerar las siguientes observaciones:

- A) Una función debe ser declarada o definida antes de ser llamada o invocada, para que el compilador pueda reconocerla en la llamada a ésta.
- B) La declaración de una función (conocida también como el prototipo de la función) consiste del tipo de retorno, el nombre de la función y la lista de parámetros terminando la sentencia con un punto y coma.
- C) Una vez que una función ha sido declarada, puede ser definida o implementada en cualquier parte del programa. La definición de la función contiene el cuerpo de la función y consiste del código que especifica lo que actualmente hace la función.
- D) Una función puede ser definida sin necesidad de ser declarada, antes del programa principal o antes de la llamada a la misma.

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- **Paso de Parámetros por Valor**
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Paso de Parámetros por Valor

El paso de parámetros por valor o por copia, significa que cuando el compilador de C/C++ compila la función y el código que invoca o llama a la función, ésta recibe una copia de los valores de los parámetros. Si dentro de la función se cambia el valor de un parámetro que viene a ser una variable local de la misma, el cambio sólo afecta a la función y no tiene efecto fuera de ella (Granizo, E., 2016).

Con el paso de parámetros por valor sólo se pueden operar con los parámetros de la función mas no modificar los contenidos de las variables que se envían a la función que son los argumentos que se copian o se asignan a los parámetros de la función. Un parámetro no es un valor por sí mismo, más bien es un valor propietario de un lugar para una variable. Esto significa que cuando la función es invocada o llamada en algún lugar del programa se envía una variable que toma el nombre del argumento que es el valor pasado dentro de la llamada de una función en particular y el parámetro de la función recibe una copia de la variable original que se envía como argumento a la misma.

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- **Funciones sin Parámetros**
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Funciones sin Parámetros

Las funciones pueden no tener parámetros, lo que significa que solamente pueden ejecutar ciertas tareas y dependiendo del tipo de retorno, devolver un valor o retornar un 'void' si la función no devuelve ningún valor (Luna, F., 2004). La función `CalcularAreaRectangulo()` del programa 5.3, es un ejemplo del uso de funciones sin parámetros, donde esta función lee el valor del ancho y del largo del rectángulo, calcula el valor del área del rectángulo y retorna el valor del cálculo del área del rectángulo, como se puede ver en la Figura 5.3.

```
float CalcularAreaRectangulo()
{
    // Declaración de variables locales.
    float ancho; // Entrada: ancho del rectángulo.
    float largo; // Entrada: largo del rectángulo.
    float area;  // Salida: área del rectángulo.

    // Imprimir un mensaje de información.
    cout << "Área de un rectángulo." << endl;
    // Leer el valor del ancho del rectángulo.
    cout << "Ingrese el valor del ancho: "; cin >> ancho;
    // Leer el valor del largo del rectángulo.
    cout << "Ingrese el valor del largo: "; cin >> largo;

    // Calcular el valor del área del rectángulo.
    area = ancho * largo;
    // Retornar el valor del área calculada.
    return area;
}
```

Figura 5.3 Ejemplo de una función sin parámetros: Zambrano M., et al., 2025.

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- **Funciones con un Parámetro**
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Funciones con Parámetros

Las funciones pueden tener un parámetro, donde el valor que se envía a la función se conoce como argumento, donde la función recibe una copia de la variable original, la cual solo puede ser utilizada, mas no modificada (Luna, F., 2004). En el programa 5.4, se muestra un ejemplo del manejo de funciones con un parámetro.

Escribir un programa utilizando funciones que calcule la quinta potencia de tres números, ingresados desde el teclado en forma seguida, utilizando la declaración y la definición de una función.

Funciones con Parámetros

```
// Librerías.
#include <iostream>
#include <cstdlib>

using namespace std;

// Declaración de las funciones (prototipos de las funciones).

float QuintaPotenciaDeUnNumero(float x);

int main()
{
    // Declaración de variables.
    float datoEntrada; // Entrada: número flotante.

    // Imprimir un mensaje de información con 2 saltos de línea.
    cout << "Quinta potencia de un número." << endl << endl;

    // Leer un número real.
    cout << "Ingrese un número real: "; cin >> datoEntrada;
    // Llamada a la función QuintaPotenciaDeUnNumero().
    cout << datoEntrada << " ^ 5 = " << QuintaPotenciaDeUnNumero(datoEntrada);
    // Imprimir un salto de línea (INTRO).
    cout << endl;

    // Leer un número real.
    cout << "Ingrese un número real: "; cin >> datoEntrada;
    // Llamada a la función QuintaPotenciaDeUnNumero().
    cout << datoEntrada << " ^ 5 = " << QuintaPotenciaDeUnNumero(datoEntrada);
    // Imprimir un salto de línea (INTRO).
    cout << endl;

    // Leer un número real.
    cout << "Ingrese un número real: "; cin >> datoEntrada;
    // Llamada a la función QuintaPotenciaDeUnNumero().
    cout << datoEntrada << " ^ 5 = " << QuintaPotenciaDeUnNumero(datoEntrada);
    // Imprimir un salto de línea (INTRO).
    cout << endl;
```

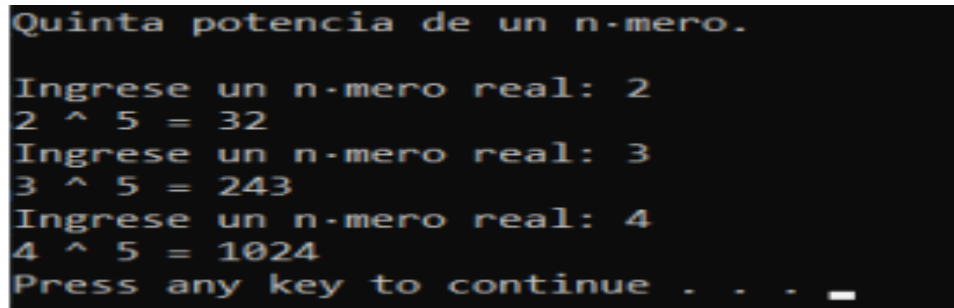
Funciones con Parámetros

```
// Incorporar una pausa en el programa.
system("pause");
return 0;
}

// Definición de las funciones (implementación de las funciones).

// Función llamada 'QuintaPotenciaDeUnNumero()' la cual tiene un parámetro
// de tipo 'float' llamado 'x' y la cual retorna un valor de tipo 'float'.
float QuintaPotenciaDeUnNumero(float x)
{
    // Declaración de variables locales.
    // Calcular el valor de  $x^5 = x * x * x * x * x$ .
    float resultado = x * x * x * x * x;
    // Retornar el valor de la variable resultado.
    return resultado;
}
```

Programa 5.4. Código del Programa. Fuente: Zambrano M., et al., 2025.



```
Quinta potencia de un n-mero.
Ingrese un n-mero real: 2
2 ^ 5 = 32
Ingrese un n-mero real: 3
3 ^ 5 = 243
Ingrese un n-mero real: 4
4 ^ 5 = 1024
Press any key to continue . . . _
```

Salida 5.4. Salida del Programa. Fuente: Zambrano M., et al., 2025.

Funciones con Parámetros

La Figura 5.4 muestra un ejemplo de cómo trabaja la función `QuintaPotenciaDeUnNumero()`, luego de ser declarada y definida. La llamada a la función `QuintaPotenciaDeUnNumero()` envía un argumento a un parámetro de la función, la misma que hace el cálculo de la quinta potencia de un número y retorna ese valor calculado a la llamada de la función. Se puede pensar que una función es una máquina separada donde se alimenta de datos de entrada (se copian los argumentos dentro de los parámetros), luego hace algo con esos parámetros (cuerpo de la función) y a la salida se obtiene un resultado (retorna algo a la llamada de la función). Las funciones permiten a los programas organizar el código en partes más manejables, donde cada parte ejecuta una tarea específica.

Funciones con Parámetros

```
void main()
{
    // Declaración de variables.
    float datoEntrada; // Entrada: número flotante.

    // Imprimir un mensaje de información con 2 INTROs.
    cout << "Quinta potencia de un número."
          << endl << endl;

    // Leer un número real.
    cout << "Ingrese un número real: ";
    cin >> datoEntrada;
    // Llamada a la función QuintaPotenciaDeUnNumero().
    cout << datoEntrada << " ^ 5 = "
          << QuintaPotenciaDeUnNumero(datoEntrada);
    // Imprimir un salto de línea (INTRO).
    cout << endl;
    // Leer un número real.
    cout << "Ingrese un número real: ";
    cin >> datoEntrada;
    // Llamada a la función QuintaPotenciaDeUnNumero().
    cout << datoEntrada << " ^ 5 = "
          << QuintaPotenciaDeUnNumero(datoEntrada);
    // Imprimir un salto de línea (INTRO).
    cout << endl;
    // Leer un número real.
    cout << "Ingrese un número real: ";
    cin >> datoEntrada;
    // Llamada a la función QuintaPotenciaDeUnNumero().
    cout << datoEntrada << " ^ 5 = "
          << QuintaPotenciaDeUnNumero(datoEntrada);
    // Imprimir un salto de línea (INTRO).
    cout << endl;

    // Incorporar una pausa en el programa.
    system("pause");
}
```

La llamada a la función hace que se pasen los argumentos dentro de los parámetros de la función. La función utiliza esta entrada para ejecutar una tarea. En este caso, la llamada a la función pasa un valor para calcular la quinta potencia de un número.

```
float QuintaPotenciaDeUnNumero(float x)
{
    float resultado = x * x * x * x * x;
    return resultado;
}
```

Cuerpo de la función que ejecuta una tarea. En este caso, se calcula la quinta potencia de 'x'.

Función que retorna un resultado de regreso a la llamada a la función. En este caso, se retorna la quinta potencia de 'x'.

Figura 5.4 Llamadas a las funciones con parámetros y retorno de resultados. Zambrano M., et al., 2025.

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- **Funciones con Varios Parámetros**
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Funciones con varios Parámetros

Las funciones pueden tener varios parámetros, ya que no están limitadas a tener cero parámetros o un solo parámetro (Luna, F., 2004). En el programa 3.5, se muestra un ejemplo del manejo de funciones con varios parámetros.

Problema 5.5: Escribir un programa utilizando funciones que lea e imprima los valores de tres vectores en el espacio, donde cada vector se compone de tres componentes (x, y, z)

```
// Librerías.
#include <iostream>
#include <cstdlib>

using namespace std;

// Declaración de las funciones (prototipos de las funciones).

void ImprimirVector3D(float x, float y, float z);

// Función principal.
int main()
{
    // Declaración de variables.
    float x; // Entrada/Salida: componente 'x' de un vector.
    float y; // Entrada/Salida: componente 'y' de un vector.
```

```
float z; // Entrada/Salida: componente 'z' de un vector.
```

```
// Leer las tres componentes de un vector 3D.
cout << "Primer Vector." << endl << endl;
cout << "Ingrese el valor de 'x': "; cin >> x;
cout << "Ingrese el valor de 'y': "; cin >> y;
cout << "Ingrese el valor de 'z': "; cin >> z;
// Imprimir un salto de línea (INTRO).
cout << endl;

cout << "V1 = ";
// Llamada a la función ImprimirVector3D(Lista_De_Parámetros).
ImprimirVector3D(x, y, z);
// Imprimir un salto de línea (INTRO).
cout << endl;

// Leer las tres componentes de un vector 3D.
cout << "Segundo Vector." << endl << endl;
cout << "Ingrese el valor de 'x': "; cin >> x;
cout << "Ingrese el valor de 'y': "; cin >> y;
cout << "Ingrese el valor de 'z': "; cin >> z;
// Imprimir un salto de línea (INTRO).
cout << endl;

cout << "V2 = ";
// Llamada a la función ImprimirVector3D(Lista_De_Parámetros).
ImprimirVector3D(x, y, z);
// Imprimir un salto de línea (INTRO).
cout << endl;

// Leer las tres componentes de un vector 3D.
cout << "Tercer Vector." << endl << endl;
cout << "Ingrese el valor de 'x': "; cin >> x;
cout << "Ingrese el valor de 'y': "; cin >> y;
cout << "Ingrese el valor de 'z': "; cin >> z;
// Imprimir un salto de línea (INTRO).
cout << endl;
```

```
// Leer las tres componentes de un vector 3D.
cout << "Tercer Vector." << endl << endl;
cout << "Ingrese el valor de 'x': "; cin >> x;
cout << "Ingrese el valor de 'y': "; cin >> y;
cout << "Ingrese el valor de 'z': "; cin >> z;
// Imprimir un salto de línea (INTRO).
cout << endl;

cout << "V3 = ";
// Llamada a la función ImprimirVector3D(Lista_De_Parámetros).
ImprimirVector3D(x, y, z);
// Imprimir un salto de línea (INTRO).
cout << endl;

// Incorporar una pausa en el programa.
system("pause");
return 0;
}

// Definición de las funciones (implementación de las funciones).

// Función llamada 'ImprimirVector3D()' la cual tiene tres parámetros
// de tipo 'float' llamados 'x', 'y', y 'z', la cual retorna un void,
// es decir, no retorna un valor y se encarga de imprimir los valores
// de las tres componentes de un vector en el espacio.
void ImprimirVector3D(float x, float y, float z)
{
    // Imprimir las componentes 'x', 'y' y 'z' del vector 3D.
    cout << "< " << x << ", " << y << ", " << z << " >" << endl;
}
```

Programa 5.5. Código del Programa. Fuente: Zambrano M., et al., 2025.

Funciones con varios Parámetros

```
Primer Vector.  
  
Ingrese el valor de 'x': 1  
Ingrese el valor de 'y': 5  
Ingrese el valor de 'z': -6  
  
V1 = < 1, 5, -6 >  
  
Segundo Vector.  
  
Ingrese el valor de 'x': 1  
Ingrese el valor de 'y': -4  
Ingrese el valor de 'z': 7  
  
V2 = < 1, -4, 7 >  
  
Tercer Vector.  
  
Ingrese el valor de 'x': 7  
Ingrese el valor de 'y': -3  
Ingrese el valor de 'z': 5  
  
V3 = < 7, -3, 5 >  
  
Press any key to continue . . .
```

Salida 5.5. Salida del Programa. Fuente: Zambrano M., et al., 2025.

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- **Funciones de la Librería de Números Aleatorios**
- Especificación del Rango de un Número Aleatorio
- Sobrecarga de Funciones

Funciones de la Librería de Números Aleatorios

- La Librería de Utilidades Generales Estándar (C Standard General Utilities Library) del lenguaje C/C++ (`#include`), proporciona una función llamada `rand()`, que puede ser utilizada para generar números pseudo-aleatorios (pseudorandom). Esta función retorna un número entero aleatorio dentro de un rango `[0, MAX_RANGO]`, donde `MAX_RANGO` es algún valor predefinido por el programador, como por ejemplo un valor constante (Dawson, M., 2008). Para generar números aleatorios diferentes cada vez que se ejecuta un programa se utiliza de manera conjunta la función `time()` con la función `srand()`, donde el valor retornado por la función `time()` se pasa como parámetro a la función `srand()`, de la siguiente manera: `srand(time(0))`; La función `time()` pertenece a la Librería de Tiempo (C Time Library) del lenguaje C/C++ (`#include`). Esta función retorna la cantidad de segundos transcurridos desde la medianoche, acorde al sistema del reloj (Stroustrup, B., 1997). La función `srand()` pertenece a la Librería de Utilidades Generales Estándar (C Standard General Utilities Library) del lenguaje C/C++ (`#include`). Esta función se utiliza como punto de entrada para generar números aleatorios conocida como la semilla generadora de números pseudo-aleatorios. A esta función se la debe llamar solamente una vez en el programa (Stroustrup, B., 1997).

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- **Especificación del Rango de un Número Aleatorio**
- Sobrecarga de Funciones

Especificación del Rango de un Número Aleatorio

- Generalmente, se utilizan números aleatorios generados dentro del rango [0, MAX_RANGO], pero hay ocasiones en las que el programador necesita generar un rango diferente, como por ejemplo el rango [3, 18]. Para este tipo de casos, se puede generalizar la obtención de un número aleatorio dentro del rango [a, b], donde el valor de 'a' representa al límite inferior del rango y el valor de 'b' representa al límite superior del rango, utilizando la siguiente fórmula propuesta por (Luna, F., 2004):

$$\text{valor} = a + \text{rand}() \% ((b + 1) - a);$$

- Así por ejemplo, si se quiere obtener números aleatorios dentro del rango [3, 18], se reemplazan dichos valores en la Ec. 5.1 y se obtiene la siguiente expresión

$$\begin{aligned}\text{valor} &= 3 + \text{rand}() \% ((18 + 1) - 3); \\ \text{valor} &= 3 + \text{rand}() \% 16;\end{aligned}$$

Contenido

- Introducción
- Objetivos
- Función
- Problema 5.1
- Estructura de una Función
- Implementación de una Función
- Problema 5.2
- Problemas que Resuelven las Funciones
- Uso de Funciones
- Paso de Parámetros por Valor
- Funciones sin Parámetros
- Funciones con un Parámetro
- Funciones con Varios Parámetros
- Funciones de la Librería de Números Aleatorios
- Especificación del Rango de un Número Aleatorio
- **Sobrecarga de Funciones**

Especificación del Rango de un Número Aleatorio

Algunas veces se requiere dos o más versiones de una misma función (Ceballos, F.J., 2007). Por ejemplo, supongamos que se requieren dos funciones que calculen el área de un rectángulo: a) La primera función `CalcularAreaRectangulo()`, no tiene parámetros, lee los datos del rectángulo y calcula el valor del área; b) La segunda función `CalcularAreaRectangulo(Lista_Parámetros)`, tiene dos parámetros, calcula el valor del área y retorna ese valor calculado, como se indican en los siguientes prototipos de las funciones sobrecargadas:

```
float CalcularAreaRectangulo();  
float CalcularAreaRectangulo(float ancho, float largo);
```

Ambas versiones, para el compilador son correctas y solo difieren en la firma de la función. La firma de la función incluye el nombre de la función y la lista de parámetros, pero no incluye el tipo de retorno

Pensamiento

Científico de la Computación

- “Todo el mundo debería saber programar”

Steve Jobs
1955-2011

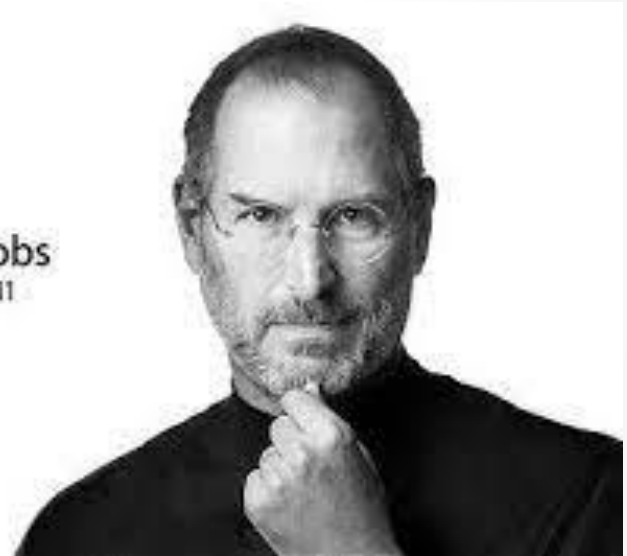


Figura 32. Universidad de las Fuerzas Armadas - ESPE.
Fuente: <https://economipedia.com/definiciones/steve-jobs.html>

¿Preguntas?

!Gracias!